

Case sheet: Production recovery under non-interactive constraints

Hyper-V differencing disk chains · v1 · September 2026

In one sentence: Two production virtual machines would not start because their virtual disk chains had broken, no usable backup existed, and I had no interactive access to the environment — so the entire repair was delivered as scripts that a local administrator executed in front of me, one shot each, with no way to intervene once they ran.

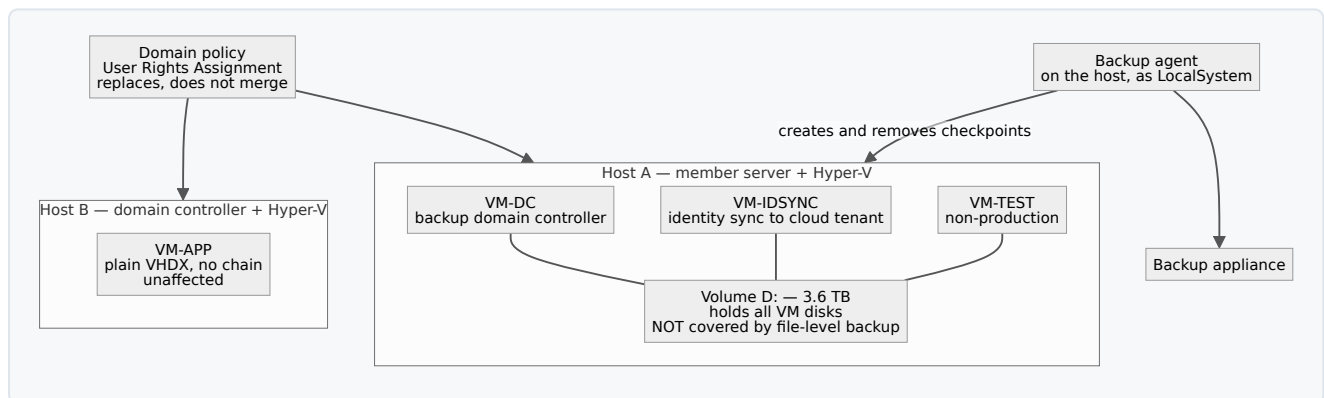
The situation

A production Hyper-V estate: two domain-critical virtual machines refused to boot. Hyper-V reported a corrupted chain — a mismatch between the identifiers of the parent virtual disk and its differencing disk. Each machine carried an orphaned checkpoint whose background merge had been left permanently pending.

The host volume holding the virtual disks was not covered by any backup. There was no restore point to fall back on. A wrong command would have destroyed domain data permanently.

Log analysis later showed the environment had been running **failed backup jobs for roughly six weeks** before anyone noticed.

The environment



Both affected machines ran a two-level chain: a dynamic parent VHDX with a single differencing AVHDX on top. The checkpoint object that should have owned that differencing disk **did not exist in the Hyper-V metadata** — an orphaned file with no record behind it. On the first machine the child disk was 4 MiB and empty; on the second it held roughly 580 MB of real data, which is why a blind rollback would have destroyed work.

The constraint

I had no connectivity to the affected hosts. Remote control and file transfer were blocked by the customer's own policy, and attachments could not pass their mail gateway. Every instruction reached the environment as **plain text pasted by a local administrator into an elevated console, with me watching over a screen share.**

That inverts the usual economics of troubleshooting. There is no iterating, no "let me just check one more thing", no undo. Whatever is handed over has to be correct on the first attempt, has to prove its own preconditions before it touches anything, and has to stop safely when reality does not match the assumption.

How it was done

- **Read-only diagnosis was kept strictly separate from intervention.** Nothing that could write was written until the picture was complete.
- **A verified copy came before any destructive step.** File-level copies of roughly **123 GB** and **93 GB** were taken and checked before the repair scripts ran.
- **Every cmdlet was verified against vendor documentation** before it was handed over, rather than trusted from memory.
- **Each script was reviewed independently before hand-off** — by a different model family than the one that drafted it — and only then approved for execution.
- **Explicit abort criteria and safety gates.** The current phase's main script carries **eight sequential gates** and a self-check before any write; on a mismatch it exits with "*STOP — nothing was changed*".
- **A non-production machine was repaired first** as a rehearsal template, so the production procedure was never the first run of anything.

How AI assistance actually worked here

The repair was produced with my own delivery framework — the same control pattern as the rest of this portfolio, applied to an incident rather than to a build. It is worth being precise about the division of labour, because the interesting claim is not "AI fixed it".

The framework is interactive, not autonomous. It never touches the customer's environment; it has no path to it. What it did was ingest the ticket history and the knowledge-base material that would otherwise have cost a day of reading, draft the diagnostic and repair scripts, and produce the analysis, the procedure and the effort estimate.

What I contributed was the control. I set the rules and the gates, decided what had to be proven before anything was allowed to write, approved every hand-off, and carried the risk of each one-shot execution. Verification was deliberately placed outside my own knowledge: every cmdlet was checked against vendor documentation rather than trusted from memory, and each artifact was reviewed by a model from a different vendor family than the one that drafted it before it was handed over.

I am not an expert in this virtualisation platform. That is the point rather than an admission. The method is built so that the quality of the outcome does not depend on how much I already knew going in — it depends on what has been verified before anything is permitted to change. Domain expertise would have made this faster. It is not what made it correct.

How it unfolded

Date	What happened	
2026-07-24	Backup job fails	symptom
2026-07-27	First machine will not start after a shutdown	symptom
2026-08-12	Second machine fails the same way	symptom
2026-08-13	Environment frozen; read-only discovery only	containment
2026-08-18	First machine recovered	recovery
2026-08-19	Second machine recovered, 580 MB preserved	recovery
2026-08-20	Appliance credential ruled out as the cause	hypothesis rejected
2026-08-21	Fault reproduced on a disposable machine	diagnosis
2026-08-24	Backup agent defect not demonstrated	hypothesis rejected
2026-08-24	Missing user right found in a domain policy	root cause
2026-08-26	Customer corrects the policy	fix
2026-08-27	Confirmed on the second host	verification
2026-08-31	Right holds across seven policy refresh cycles	verification
from 2026-09	Phase two: remaining chain and working backups	in progress

Result

Both production virtual machines were recovered. The incident record reads **"Data loss: None / Restore from backup required: No"** — the empty 4 MiB differencing disk on the first machine was discarded, while roughly **580-608 MB** of genuine data in the second machine's child disk was preserved rather than rolled back.

Phase one consumed **13.5 hours** across two recovery sessions. Phase two — repairing the remaining chain and restoring working backups — is running now; its main script has been through **seven revisions plus independent review rounds**.

The safety design has since been proven in production rather than asserted: during a customer-run execution, a gate detected a mismatch and stopped the script before any change was made. That is the outcome a gate exists for.

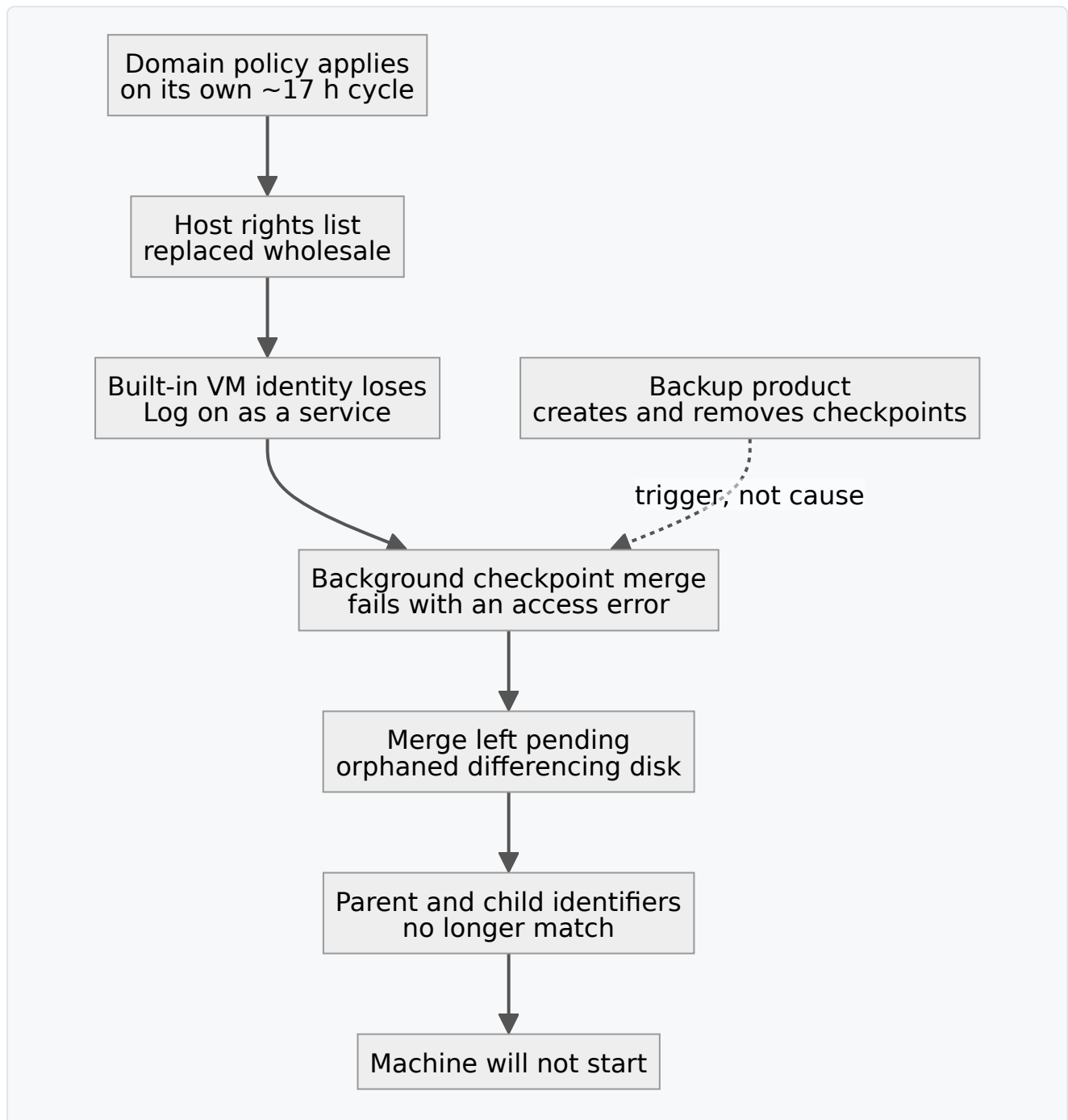
The root cause, and what it was not

The obvious suspect was the third-party backup product: the corruption always appeared around its checkpoint activity. The evidence pointed somewhere else.

A domain group policy was overwriting the *Log on as a service* user right on the hosts. Because that assignment replaces rather than merges, and the domain list omitted the built-in Hyper-V virtual machine identity, the right silently disappeared from the hosts. The background checkpoint merge then failed with an access error, leaving the merge pending and eventually breaking the parent-child identifier binding — at which point the machine would no longer start. The same configuration defect was present on **both** hosts.

The backup product was the trigger, in that it created and removed the checkpoints on which the missing right mattered. **A defect in the product itself was never demonstrated, and the write-up says so.** Naming a vendor as the cause would have been the comfortable conclusion; the evidence did not support it, so it was not written.

The failure mechanism



The customer corrected the policy. The fix was then verified as durable across **five days and seven policy refresh cycles** before the case was closed.

Why it matters

Three things generalise beyond this incident.

Backup that is running is not backup that works. Six weeks of failing jobs went unobserved, and the volume that mattered was outside the backup scope entirely. Both facts were only discovered because something else broke.

Non-interactive delivery is a discipline, not a handicap. Being unable to improvise forces the properties that ought to be there anyway: preconditions checked in the script, rehearsal on a disposable target, independent review before hand-off, and a defined stop.

The first plausible cause is not the cause. The strength of a root-cause analysis is measured by what it refuses to claim.

Method can substitute for domain knowledge, within limits. It compensates for what you do not know; it does not tell you which property matters when you do not know the domain well enough to ask. Here the failure was legible — error codes, documented chain semantics, a disposable target to rehearse on — and the method could therefore carry it. That condition is worth checking before claiming the same anywhere else.

Customer, location, hostnames, domain, vendor names, ticket references and policy identifiers are omitted. Full engagement records are available for technical review under NDA.